

CSC311H5 Introduction to Machine Learning

Lecture 12

University of Toronto Mississauga

Unsupervised Learning

In the last two lectures, we explored approaches for **unsupervised Learning**

No labeled examples; instead, looking for *interesting patterns* in the data

But what are *interesting patterns*?

Unsupervised Learning

In the last two lectures, we explored approaches for **unsupervised Learning**

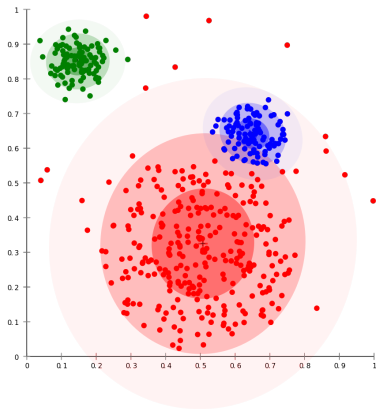
No labeled examples; instead, looking for *interesting patterns* in the data

But what are *interesting patterns*?

- Do data form *clusters*: i.e., are there groups of data points that are more similar to each other than to other data points?
- Can data be expressed in a lower-dimensional space?: i.e., do data points tend to fall in a *subspace* or *manifold* of the space that they inhabit?

Clustering

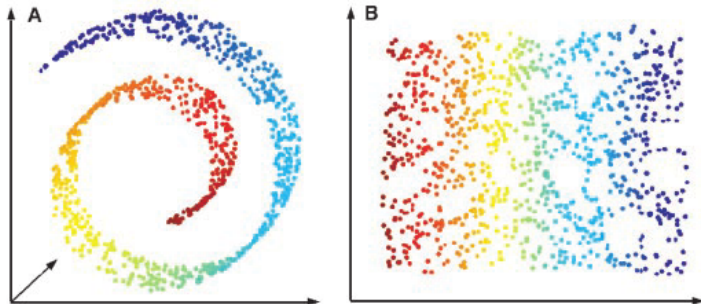
Clustering: Do data form *clusters*: i.e., are there groups of data points that are more similar to each other than to other data points?



Dimensionality Reduction

Dimensionality Reduction Can data be expressed in a lower-dimensional space?: i.e., do data points tend to fall in a *subspace* or *manifold* of the space that they inhabit?

After IsoMap



Dimensionality Reduction

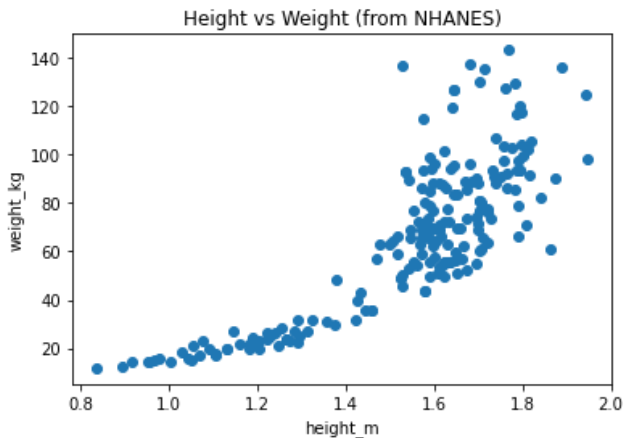
We want to represent data in a new coordinate system with fewer dimensions, while preserving as much *information* as possible

Why?

- We can't easily visualize high dimensional data, but can easily plot 2D (and 3D data)
- We want to extract meaningful features from the data
- We want to compress the data while preserving most of the information

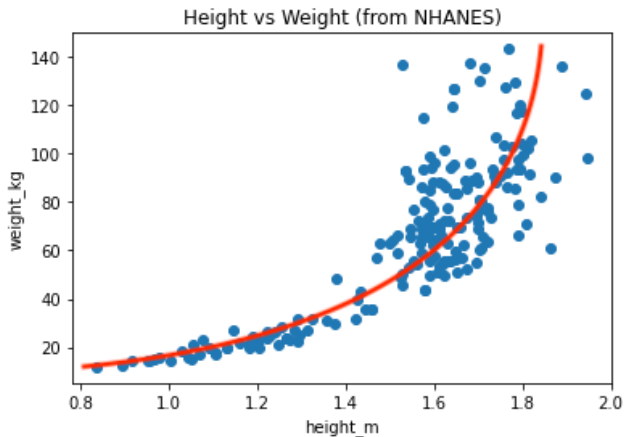
Example: NHANES Data

Suppose we have the following 2D data that we wish to reduce to 1D:



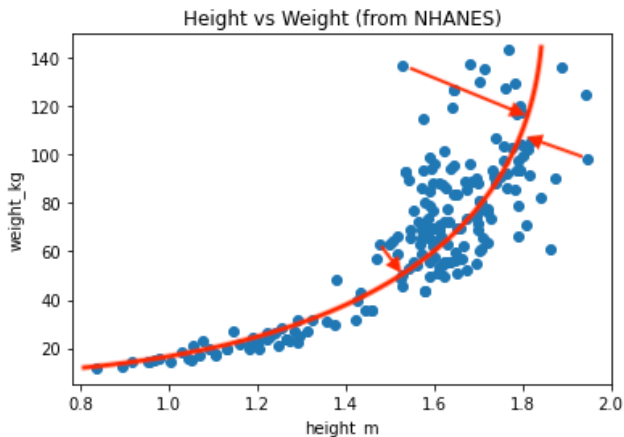
Example: NHANES Data: Manifold

This is likely possible, because the data lies (roughly) along a 1D manifold.



Example: NHANES Data: Projection

We can then *project the 2D data onto the 1D manifold*

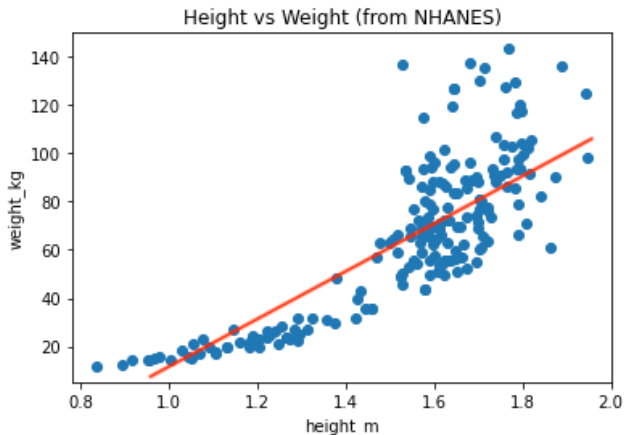


Now, instead of storing/using two features (height/weight), we store a single feature: the position of our data point along the 1D manifold.

Principle Component Analysis

Today, we will discuss a simpler form of dimensionality reduction: **linear dimensionality reduction**

Since PCA is a linear model, this mapping will be a **linear projection**.



Section 1

Principal Component Analysis

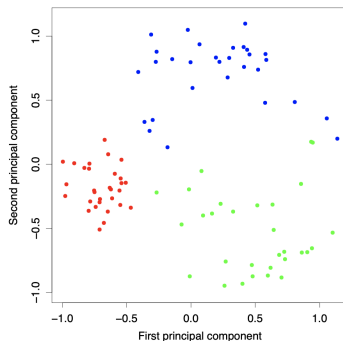
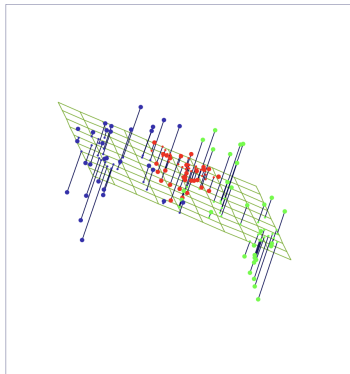
Principal Component Analysis

PCA is a **linear** dimensionality reduction technique

The transformed data is a linear transformation or a **linear projection** of the original data

We want to find a hyperplane that the data lies in and project the data onto that hyperplane.

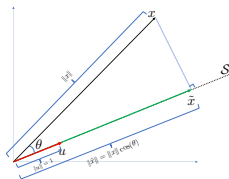
PCA 3D to 2D



From the Elements of Statistical Learning

Euclidean Projection onto a 1D subspace

How to project onto a 1-dimensional subspace?



- Subspace $\mathcal{S}$ is the line along the unit vector $\mathbf{u}$; here, $\mathbf{u}$ is a **basis** for $\mathcal{S}$
 - ▶ any point in $\mathcal{S}$ can be written as $z\mathbf{u}$ for some $z \in \mathbb{R}$.
- Projection of $\mathbf{x}$ on $\mathcal{S}$ is denoted by $\text{Proj}_{\mathcal{S}}(\mathbf{x})$
- Recall: $\mathbf{x}^\top \mathbf{u} = \|\mathbf{x}\| \|\mathbf{u}\| \cos(\theta) = \|\mathbf{x}\| \cos(\theta)$
- $\text{Proj}_{\mathcal{S}}(\mathbf{x}) = \underbrace{\mathbf{x}^\top \mathbf{u}}_{\text{length of proj}} \cdot \underbrace{\mathbf{u}}_{\text{direction of proj}} = \|\tilde{\mathbf{x}}\| \mathbf{u}$

Euclidean Projection onto a K -D subspace

How to project onto a K -dimensional subspace?

- **Idea:** choose an orthonormal basis $\{\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_K\}$ for $\mathcal{S}$
- Project onto each unit vector individually (as in previous slide), and sum together the projections.

Mathematically, the projection is given as:

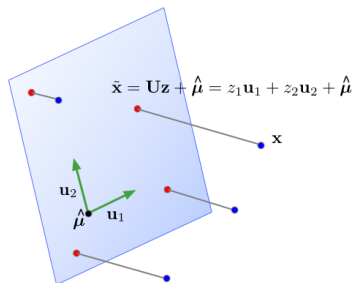
$$\text{Proj}_{\mathcal{S}}(\mathbf{x}) = \sum_{i=1}^K z_i \mathbf{u}_i \quad \text{where} \quad z_i = \mathbf{x}^\top \mathbf{u}_i$$

In vector form:

$$\text{Proj}_{\mathcal{S}}(\mathbf{x}) = \mathbf{U}\mathbf{z} \quad \text{where} \quad \mathbf{z} = \mathbf{U}^\top \mathbf{x}$$

Projection onto a Subspace

So far, we assumed the subspace passes through $\mathbf{0}$. But the “subspaces” we want to project onto are really **affine spaces**, and can have an arbitrary origin.



$$\mathbf{z} = \mathbf{U}^\top (\mathbf{x} - \hat{\boldsymbol{\mu}})$$

- $\tilde{\mathbf{x}}$ is also called the **reconstruction** of $\mathbf{x}$.
- $\mathbf{z}$ is its **representation**, or **code**.

Projection Setup and Terminology

If we have a K -dimensional subspace in a D -dimensional input space, then $\mathbf{x} \in \mathbb{R}^D$ and $\mathbf{z} \in \mathbb{R}^K$.

If the data points $\mathbf{x}$ all lie close to their reconstructions $\tilde{\mathbf{x}}$, then we can approximate distances, etc. in terms of these same operations on the code vectors $\mathbf{z}$.

If $K \ll D$, then it's much cheaper to work with $\mathbf{z}$ than $\mathbf{x}$.

Projection Setup and Terminology

If we have a K -dimensional subspace in a D -dimensional input space, then $\mathbf{x} \in \mathbb{R}^D$ and $\mathbf{z} \in \mathbb{R}^K$.

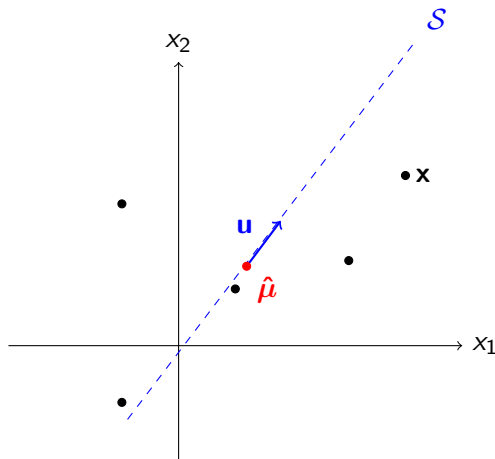
If the data points $\mathbf{x}$ all lie close to their reconstructions $\tilde{\mathbf{x}}$, then we can approximate distances, etc. in terms of these same operations on the code vectors $\mathbf{z}$.

If $K \ll D$, then it's much cheaper to work with $\mathbf{z}$ than $\mathbf{x}$.

A mapping to a space that's easier to manipulate or visualize is called a **representation**, and learning such a mapping is **representation learning**.

Example Projection Exercise

Exercise: What is the projection of $\mathbf{x}$ onto $\mathcal{S}$? Compute the code representation/code z . Here, z is a scalar since $\mathcal{S}$ is 1D.



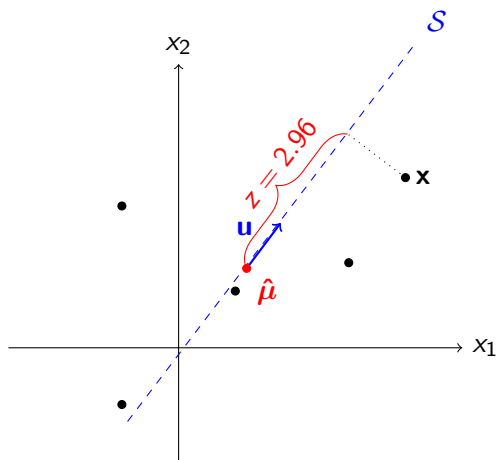
$$\mathbf{u} = \begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

$$\hat{\boldsymbol{\mu}} = \begin{bmatrix} 1.2 \\ 1.4 \end{bmatrix}$$

$$z = \mathbf{u}^T (\mathbf{x} - \hat{\boldsymbol{\mu}}) \\ = \dots$$

Example Projection Exercise Solutions



$$\mathbf{u} = \begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix}$$

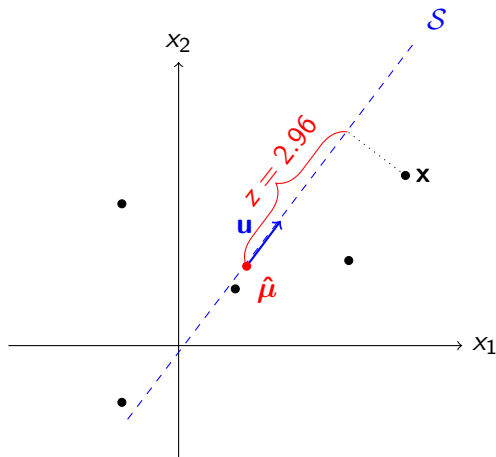
$$\mathbf{x} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

$$\hat{\boldsymbol{\mu}} = \begin{bmatrix} 1.2 \\ 1.4 \end{bmatrix}$$

$$\begin{aligned} z &= \mathbf{u}^T (\mathbf{x} - \hat{\boldsymbol{\mu}}) \\ &= 0.6(4 - 1.2) \\ &\quad + 0.8(3 - 1.4) \\ &= 2.96 \end{aligned}$$

Example Projection Reconstruction Exercise

Exercise: What is the reconstruction $\tilde{\mathbf{x}}$?



$$\mathbf{u} = \begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix}$$

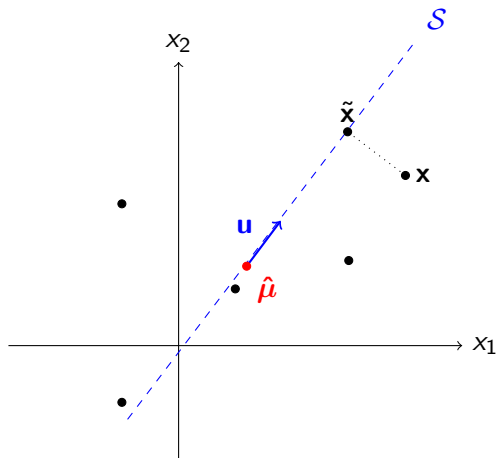
$$\mathbf{x} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

$$\hat{\boldsymbol{\mu}} = \begin{bmatrix} 1.2 \\ 1.4 \end{bmatrix}$$

$$z = \mathbf{u}^T (\mathbf{x} - \hat{\boldsymbol{\mu}}) = 2.96$$

$$\tilde{\mathbf{x}} = \hat{\boldsymbol{\mu}} + z\mathbf{u}$$

Example Projection Reconstruction Solution



$$\mathbf{u} = \begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

$$\hat{\boldsymbol{\mu}} = \begin{bmatrix} 1.2 \\ 1.4 \end{bmatrix}$$

$$z = \mathbf{u}^T (\mathbf{x} - \hat{\boldsymbol{\mu}}) = 2.96$$

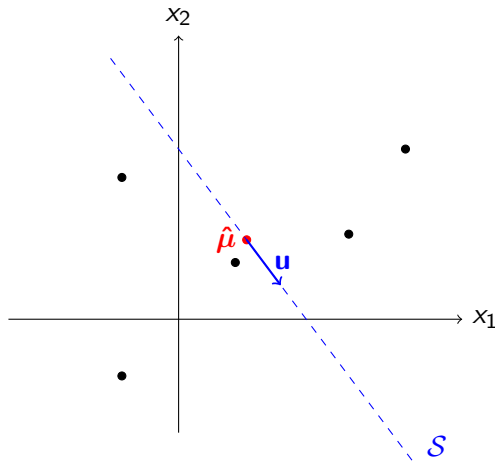
$$\tilde{\mathbf{x}} = \hat{\boldsymbol{\mu}} + z\mathbf{u}$$

$$= \begin{bmatrix} 1.2 \\ 1.4 \end{bmatrix} + 2.96 \begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix}$$

$$= \begin{bmatrix} 2.98 \\ 3.77 \end{bmatrix}$$

Example Projection Along a Different Direction

Consider projection along a different axes $\mathbf{u}$.



Intuitively, is this a **better** or **worse** subspace to project our data onto?

Learning a Good Subspace

How do we choose a good subspace $\mathcal{S}$?

- Origin $\hat{\mu}$ is the empirical mean of the data
- Need to choose a $D \times K$ matrix $\mathbf{U}$ with orthonormal columns.

Two reasonable criteria:

- Minimize the **reconstruction error**:

$$\min_{\mathbf{U}} \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)}\|^2$$

- Maximize the **variance of the reconstruction**:

$$\max_{\mathbf{U}} \frac{1}{N} \sum_{i=1}^N \|\tilde{\mathbf{x}}^{(i)} - \hat{\mu}\|^2$$

Reconstruction Error vs Variance

Claim: These two criteria are actually equivalent!

$$\frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)}\|^2 = \text{const} - \frac{1}{N} \sum_{i=1}^N \|\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}}\|^2$$

Recall that

- $\tilde{\mathbf{x}}^{(i)} = \hat{\boldsymbol{\mu}} + \mathbf{U}\mathbf{z}^{(i)}$
- $\mathbf{z}^{(i)} = \mathbf{U}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})$

What about the variance of the representation?

What about this third possible criteria?

- Maximize the **variance of the representation**:

$$\max_{\mathbf{U}} \frac{1}{N} \sum_{i=1}^N \| \mathbf{z}^{(i)} \|^2$$

What about the variance of the representation?

What about this third possible criteria?

- Maximize the **variance of the representation**:

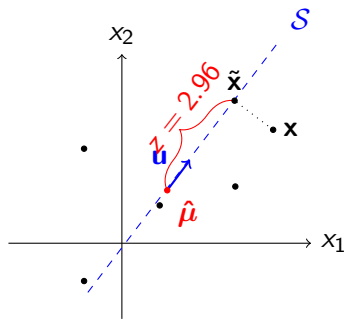
$$\max_{\mathbf{U}} \frac{1}{N} \sum_{i=1}^N \|\mathbf{z}^{(i)}\|^2$$

Because the columns of $\mathbf{U}$ are orthogonal, $\mathbf{U}^\top \mathbf{U} = \mathbf{I}$, the norm of centered reconstruction is equal to norm of representation:

$$\|\tilde{\mathbf{x}} - \hat{\boldsymbol{\mu}}\|^2 = \|\mathbf{U}\mathbf{z}\|^2 = \mathbf{z}^\top \mathbf{U}^\top \mathbf{U} \mathbf{z} = \mathbf{z}^\top \mathbf{z} = \|\mathbf{z}\|^2$$

Variance of the Representation vs. Reconstruction

Straightforward to show geometrically that $\|\tilde{\mathbf{x}} - \hat{\boldsymbol{\mu}}\|^2 = \|\mathbf{z}\|^2$:



$\|\mathbf{z}\|^2$ here is the length of the red portion in $\mathcal{S}$.

$\|\tilde{\mathbf{x}} - \hat{\boldsymbol{\mu}}\|^2$ here is the length of the vector $\tilde{\mathbf{x}} - \hat{\boldsymbol{\mu}}$ in $\mathbb{R}^2$.

Thus, the variance of reconstructions is equal to variance of code vectors:

$$\frac{1}{N} \sum_{i=1}^N \|\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}}\|^2 = \frac{1}{N} \sum_{i=1}^N \|\mathbf{z}^{(i)}\|^2$$

Reconstruction Error vs Variance: Pythagorean Theorem

Our goal is to show that $\frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)}\|^2 = \text{const} - \frac{1}{N} \sum_{i=1}^N \|\tilde{\mathbf{x}} - \hat{\boldsymbol{\mu}}\|^2$

Claim: The vectors $\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}}$ and $\tilde{\mathbf{x}}^{(i)} - \mathbf{x}^{(i)}$ are **orthogonal** (and we can thus apply the Pythagorean theorem).

Proof: First, we can write $\tilde{\mathbf{x}}^{(i)} = \hat{\boldsymbol{\mu}} + \mathbf{U}\mathbf{U}^\top(\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})$.

Reconstruction Error vs Variance: Pythagorean Theorem

Our goal is to show that $\frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)}\|^2 = \text{const} - \frac{1}{N} \sum_{i=1}^N \|\tilde{\mathbf{x}} - \hat{\boldsymbol{\mu}}\|^2$

Claim: The vectors $\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}}$ and $\tilde{\mathbf{x}}^{(i)} - \mathbf{x}^{(i)}$ are **orthogonal** (and we can thus apply the Pythagorean theorem).

Proof: First, we can write $\tilde{\mathbf{x}}^{(i)} = \hat{\boldsymbol{\mu}} + \mathbf{U}\mathbf{U}^\top(\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})$.

We know that two vectors $\mathbf{a}, \mathbf{b}$ are orthogonal $\iff \mathbf{a}^\top \mathbf{b} = 0$.

Reconstruction Error vs Variance: Pythagorean Theorem

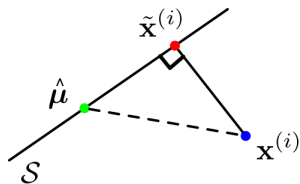
Our goal is to show that $\frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)}\|^2 = \text{const} - \frac{1}{N} \sum_{i=1}^N \|\tilde{\mathbf{x}} - \hat{\boldsymbol{\mu}}\|^2$

Claim: The vectors $\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}}$ and $\tilde{\mathbf{x}}^{(i)} - \mathbf{x}^{(i)}$ are **orthogonal** (and we can thus apply the Pythagorean theorem).

Proof: First, we can write $\tilde{\mathbf{x}}^{(i)} = \hat{\boldsymbol{\mu}} + \mathbf{U}\mathbf{U}^\top(\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})$.

We know that two vectors $\mathbf{a}, \mathbf{b}$ are orthogonal $\iff \mathbf{a}^\top \mathbf{b} = 0$.

So we compute $(\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}})^\top (\tilde{\mathbf{x}}^{(i)} - \mathbf{x}^{(i)})$:



$$\begin{aligned} & (\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}})^\top (\tilde{\mathbf{x}}^{(i)} - \mathbf{x}^{(i)}) \\ &= (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})^\top \mathbf{U}\mathbf{U}^\top (\hat{\boldsymbol{\mu}} - \mathbf{x}^{(i)} + \mathbf{U}\mathbf{U}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})) \end{aligned}$$

Reconstruction Error vs Variance: Pythagorean Theorem

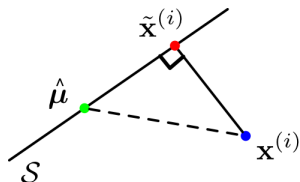
Our goal is to show that $\frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)}\|^2 = \text{const} - \frac{1}{N} \sum_{i=1}^N \|\tilde{\mathbf{x}} - \hat{\boldsymbol{\mu}}\|^2$

Claim: The vectors $\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}}$ and $\tilde{\mathbf{x}}^{(i)} - \mathbf{x}^{(i)}$ are **orthogonal** (and we can thus apply the Pythagorean theorem).

Proof: First, we can write $\tilde{\mathbf{x}}^{(i)} = \hat{\boldsymbol{\mu}} + \mathbf{U}\mathbf{U}^\top(\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})$.

We know that two vectors $\mathbf{a}, \mathbf{b}$ are orthogonal $\iff \mathbf{a}^\top \mathbf{b} = 0$.

So we compute $(\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}})^\top (\tilde{\mathbf{x}}^{(i)} - \mathbf{x}^{(i)})$:



$$\begin{aligned} & (\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}})^\top (\tilde{\mathbf{x}}^{(i)} - \mathbf{x}^{(i)}) \\ &= (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})^\top \mathbf{U}\mathbf{U}^\top (\hat{\boldsymbol{\mu}} - \mathbf{x}^{(i)} + \mathbf{U}\mathbf{U}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})) \\ &= (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})^\top \mathbf{U}\mathbf{U}^\top (\hat{\boldsymbol{\mu}} - \mathbf{x}^{(i)}) + (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})^\top \mathbf{U}\mathbf{U}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}) \end{aligned}$$

Reconstruction Error vs Variance: Pythagorean Theorem

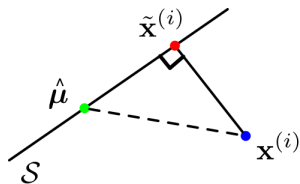
Our goal is to show that $\frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)}\|^2 = \text{const} - \frac{1}{N} \sum_{i=1}^N \|\tilde{\mathbf{x}} - \hat{\boldsymbol{\mu}}\|^2$

Claim: The vectors $\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}}$ and $\tilde{\mathbf{x}}^{(i)} - \mathbf{x}^{(i)}$ are **orthogonal** (and we can thus apply the Pythagorean theorem).

Proof: First, we can write $\tilde{\mathbf{x}}^{(i)} = \hat{\boldsymbol{\mu}} + \mathbf{U}\mathbf{U}^\top(\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})$.

We know that two vectors $\mathbf{a}, \mathbf{b}$ are orthogonal $\iff \mathbf{a}^\top \mathbf{b} = 0$.

So we compute $(\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}})^\top (\tilde{\mathbf{x}}^{(i)} - \mathbf{x}^{(i)})$:



$$\begin{aligned} & (\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}})^\top (\tilde{\mathbf{x}}^{(i)} - \mathbf{x}^{(i)}) \\ &= (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})^\top \mathbf{U}\mathbf{U}^\top (\hat{\boldsymbol{\mu}} - \mathbf{x}^{(i)} + \mathbf{U}\mathbf{U}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})) \\ &= (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})^\top \mathbf{U}\mathbf{U}^\top (\hat{\boldsymbol{\mu}} - \mathbf{x}^{(i)}) + (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})^\top \mathbf{U}\mathbf{U}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}) \\ &= 0 \end{aligned}$$

Applying the Pythagorean Theorem

The Pythagorean Theorem tells us:

$$\|\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}}\|^2 + \|\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)}\|^2 = \|\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}\|^2 \quad \text{for each } i$$

By averaging over data, we obtain

$$\begin{aligned} & \underbrace{\frac{1}{N} \sum_{i=1}^N \|\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}}\|^2}_{\text{projected variance}} + \underbrace{\frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)}\|^2}_{\text{reconstruction error}} \\ &= \underbrace{\frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}\|^2}_{\text{constant}} \end{aligned}$$

Therefore, **projected variance = constant - reconstruction error**

Maximizing the variance is equivalent to minimizing the reconstruction error!

Principle Component Analysis

Choosing a subspace to maximize the projected variance, or minimize the reconstruction error, is called **principal component analysis (PCA)**.

But how can we find projection directions **$\mathbf{U}$** to **maximize the projected variance**?

Section 2

Maximizing Projected Variance

Principle Component Analysis

Choosing a subspace to maximize the projected variance, or minimize the reconstruction error, is called **principal component analysis (PCA)**.

But how can we find projection directions **$\mathbf{U}$** to **maximize the projected variance**?

Principle Component Analysis

Choosing a subspace to maximize the projected variance, or minimize the reconstruction error, is called **principal component analysis (PCA)**.

But how can we find projection directions **U** to **maximize the projected variance**?

Consider the **empirical covariance matrix**:

$$\hat{\Sigma} = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}^{(i)} - \hat{\mu})(\mathbf{x}^{(i)} - \hat{\mu})^\top$$

Recall:

- $\hat{\Sigma}$ is symmetric and positive semidefinite.
- Can be written $\Sigma = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ where $\mathbf{\Lambda}$ is diagonal and $\mathbf{Q}$ is orthogonal. The diagonals of $\mathbf{\Lambda}$ are the eigenvalues. The columns of $\mathbf{Q}$ are eigenvectors of Σ .

Optimal PCA Subspace

Consider the **empirical covariance matrix**:

$$\hat{\Sigma} = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}^{(i)} - \hat{\mu})(\mathbf{x}^{(i)} - \hat{\mu})^\top = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$$

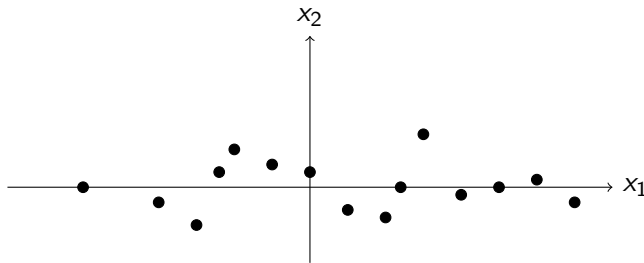
The optimal PCA subspace is spanned by the top K eigenvectors of $\hat{\Sigma}$!

- More precisely, choose the first K of any orthonormal eigenbasis for $\hat{\Sigma}$.
- The general case is tricky, but we'll show this for $K = 1$.

These eigenvectors are called **principal components**, analogous to the principal axes of an ellipse.

Intuition: Diagonal $\hat{\Sigma}$

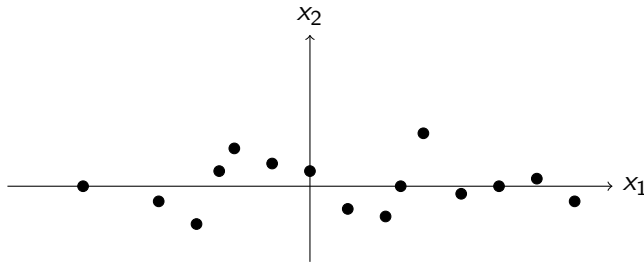
Suppose that we have a data set with $\hat{\Sigma} = \begin{bmatrix} 5 & 0 \\ 0 & 1 \end{bmatrix}$



What would be the optimal PCA direction $\mathbf{u}$?

Intuition: Diagonal $\hat{\Sigma}$

Suppose that we have a data set with $\hat{\Sigma} = \begin{bmatrix} 5 & 0 \\ 0 & 1 \end{bmatrix}$

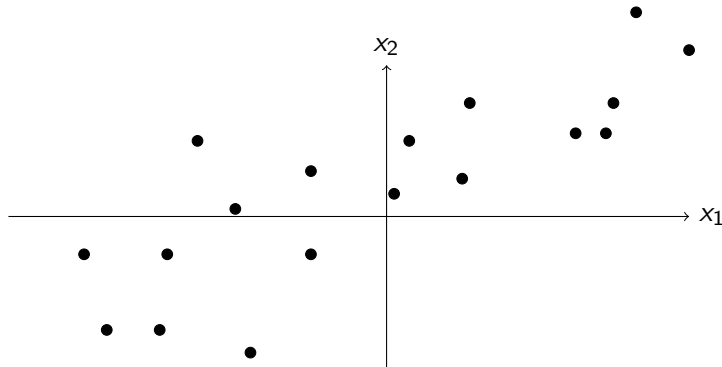


What would be the optimal PCA direction $\mathbf{u}$?

This direction is exactly the eigenvector of $\hat{\Sigma}$ with the largest eigenvalue!

Intuition: General $\hat{\Sigma}$

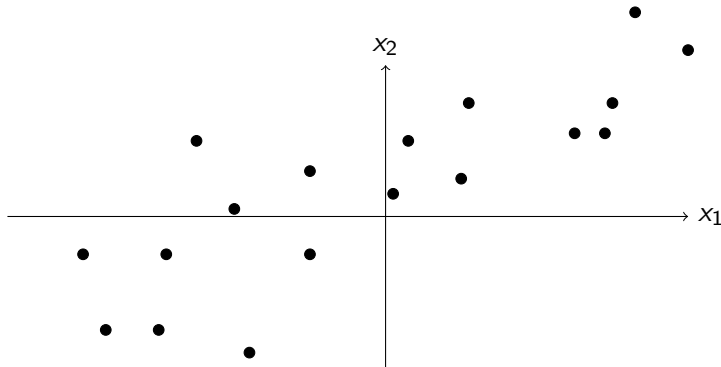
Suppose that we have a data set with $\hat{\Sigma} \approx \begin{bmatrix} 5 & 0.4 \\ 0.4 & 1 \end{bmatrix}$



What would be the optimal PCA direction $\mathbf{u}$?

Intuition: General $\hat{\Sigma}$

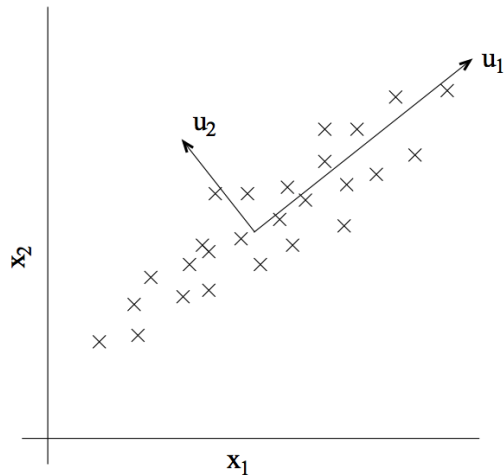
Suppose that we have a data set with $\hat{\Sigma} \approx \begin{bmatrix} 5 & 0.4 \\ 0.4 & 1 \end{bmatrix}$



What would be the optimal PCA direction $\mathbf{u}$?

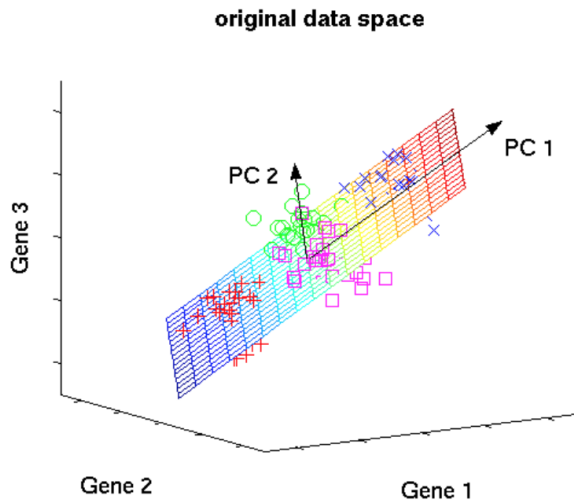
This direction is, again, exactly the eigenvector of $\hat{\Sigma}$ with the largest eigenvalue!

PCA Dimensions (2D)



See lecture 10b.

PCA Dimensions (3D)



Deriving PCA ($K=1$)

For $K = 1$, we are fitting a unit vector $\mathbf{u}$, and the code is a scalar $z^{(i)} = \mathbf{u}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})$.

$$\frac{1}{N} \sum_i \|\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}}\|^2 = \frac{1}{N} \sum_i [z^{(i)}]^2 = \frac{1}{N} \sum_i (\mathbf{u}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}))^2$$

Deriving PCA ($K=1$)

For $K = 1$, we are fitting a unit vector $\mathbf{u}$, and the code is a scalar $z^{(i)} = \mathbf{u}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})$.

$$\frac{1}{N} \sum_i \|\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}}\|^2 = \frac{1}{N} \sum_i [z^{(i)}]^2 = \frac{1}{N} \sum_i (\mathbf{u}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}))^2$$

Deriving PCA ($K=1$)

For $K = 1$, we are fitting a unit vector $\mathbf{u}$, and the code is a scalar $z^{(i)} = \mathbf{u}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})$.

$$\begin{aligned} \frac{1}{N} \sum_i \|\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}}\|^2 &= \frac{1}{N} \sum_i [z^{(i)}]^2 = \frac{1}{N} \sum_i (\mathbf{u}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}))^2 \\ &= \frac{1}{N} \sum_{i=1}^N \mathbf{u}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}) (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})^\top \mathbf{u} \end{aligned}$$

Deriving PCA ($K=1$)

For $K = 1$, we are fitting a unit vector $\mathbf{u}$, and the code is a scalar $z^{(i)} = \mathbf{u}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})$.

$$\begin{aligned}\frac{1}{N} \sum_i \|\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}}\|^2 &= \frac{1}{N} \sum_i [z^{(i)}]^2 = \frac{1}{N} \sum_i (\mathbf{u}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}))^2 \\ &= \frac{1}{N} \sum_{i=1}^N \mathbf{u}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}) (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})^\top \mathbf{u} \\ &= \mathbf{u}^\top \left[\frac{1}{N} \sum_{i=1}^N (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}) (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})^\top \right] \mathbf{u}\end{aligned}$$

Deriving PCA (K=1)

For $K = 1$, we are fitting a unit vector $\mathbf{u}$, and the code is a scalar $z^{(i)} = \mathbf{u}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})$.

$$\begin{aligned}\frac{1}{N} \sum_i \|\tilde{\mathbf{x}}^{(i)} - \hat{\boldsymbol{\mu}}\|^2 &= \frac{1}{N} \sum_i [z^{(i)}]^2 = \frac{1}{N} \sum_i (\mathbf{u}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}))^2 \\&= \frac{1}{N} \sum_{i=1}^N \mathbf{u}^\top (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}) (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})^\top \mathbf{u} \\&= \mathbf{u}^\top \left[\frac{1}{N} \sum_{i=1}^N (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}) (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}})^\top \right] \mathbf{u} \\&= \mathbf{u}^\top \hat{\boldsymbol{\Sigma}} \mathbf{u} \\&= \mathbf{u}^\top \mathbf{Q} \boldsymbol{\Lambda} \mathbf{Q}^\top \mathbf{u} && \text{Spectral Decomposition} \\&= \mathbf{a}^\top \boldsymbol{\Lambda} \mathbf{a} && \text{for } \mathbf{a} = \mathbf{Q}^\top \mathbf{u} \\&= \sum_{j=1}^D \lambda_j a_j^2\end{aligned}$$

Deriving PCA (K=1)

So, we need to choose $\mathbf{a} = \mathbf{Q}^\top \mathbf{u}$ that maximizes

$$\mathbf{a}^\top \mathbf{\Lambda} \mathbf{a} = \sum_{j=1}^D \lambda_j a_j^2$$

Assume that $\mathbf{\Lambda}$ has the eigenvalues in sorted order: $\lambda_1 \geq \lambda_2 \geq \dots$

Since $\mathbf{u}$ is a unit vector, then $\mathbf{a}$ is also a unit vector, so $\sum_{j=1}^D a_j^2 = 1$.

Quantity $\sum_{j=1}^D \lambda_j a_j^2$ is maximized by selecting the largest eigenvalue!

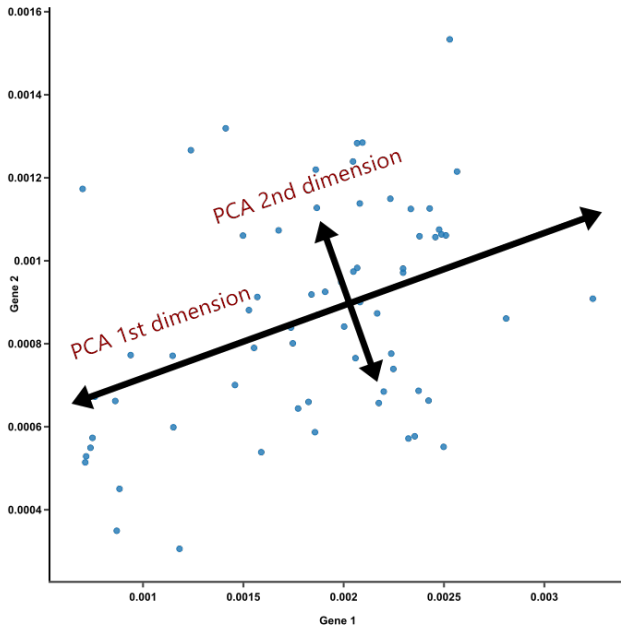
So $\mathbf{u} = \mathbf{Q} \mathbf{a} = \mathbf{q}_1$, the eigenvector corresponding to the largest eigenvalue.

Deriving PCA

Summary: The first PCA direction is the eigenvector corresponding to the largest eigenvalue.

A similar argument shows that the k th principal component is the k th eigenvector of $\hat{\Sigma}$.

PCA as Rotation



Decorrelation

Interesting fact: The dimensions of the code $\mathbf{z}$ are *decorrelated*. Assume that $\hat{\boldsymbol{\mu}} = \mathbf{0}$, then:

$$\hat{\boldsymbol{\Sigma}} = \mathbf{Q}\boldsymbol{\Lambda}\mathbf{Q}^\top$$

$$\mathbf{X}^\top \mathbf{X} = \mathbf{Q}\boldsymbol{\Lambda}\mathbf{Q}^\top$$

$$\mathbf{Q}^\top \mathbf{X}^\top \mathbf{X} \mathbf{Q} = \boldsymbol{\Lambda}$$

$$(\mathbf{X}\mathbf{Q})^\top (\mathbf{X}\mathbf{Q}) = \boldsymbol{\Lambda}$$

So if we rotate $\mathbf{X}$ using $\mathbf{Q}$, the covariance of the transformed data will be diagonal!

The columns of $\mathbf{X}\mathbf{Q}$ are uncorrelated.

Section 3

Closing Thoughts

What this course focused on:

Supervised learning: regression, classification

- Choose model, loss function, optimizer
- Parametric vs. nonparametric
- Generative vs. discriminative
- Iterative optimization vs. closed-form solutions

Unsupervised learning: dimensionality reduction and clustering

CSC413 Teaser: Neural Nets

CSC311 covered some fundamental ideas, most of which are more than 20-30 years old.

Big shift of the past decade: neural nets and deep learning

- 2010: neural nets significantly improved speech recognition accuracy (after 20 years of stagnation)
- 2012–2015: neural nets reduced error rates for object recognition by a factor of 6
- 2016: a program called AlphaGo defeated the human Go champion
- 2015–2018: neural nets learned to produce convincing high-resolution images
- 2018–today: large language models a sophisticated ability to generate natural language text and learn from few examples

CSC413 Teaser: Beyond Scalar/Discrete Targets

CSC311 focused on regression and classification, i.e. scalar-valued or discrete outputs

That only covers a small fraction of use cases. Often, we want to output something more structured:

- text (e.g. image captioning, machine translation)
- dense labels of images (e.g. semantic segmentation)
- graphs (e.g. molecule design)

This used to be known as structured prediction, but now it's so routine we don't need a name for it.

CSC413 Teaser: Representation Learning

CSC311 talked about neural nets as learning feature maps you can use for regression/classification

More generally, want to learn a representation of the data such that mathematical operations on the representation are semantically meaningful

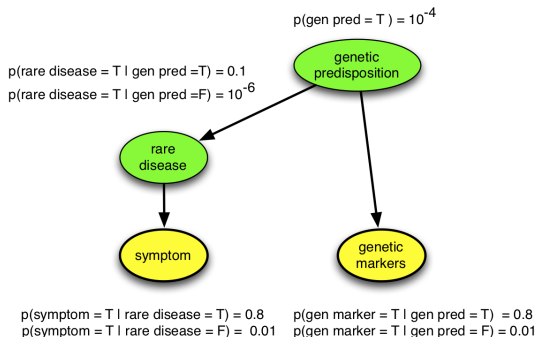
Classic (decades-old) example: representing words as vectors

- Measure semantic similarity using the dot product between word vectors (or dissimilarity using Euclidean distance)
- Represent a web page with the average of its word vectors

CSC412 Teaser: Probabilistic Graphical Models

CSC311 scratched the surface of probabilistic models.

Probabilistic graphical models (PGMs) let you encode complex probabilistic relationships between lots of variables.



Ghahramani, 2015, "Probabilistic ML and artificial intelligence"

CSC412 Teaser: PGM Inference

In CSC311, we derived inference methods by inspection for some easy special cases (e.g. GDA, naive Bayes)

In CSC412, you'll learn much more general and powerful inference techniques that expand the range of models you can build

- Exact inference using dynamic programming, for certain types of graph structures (e.g. $\sim$ chains)
- Markov chain Monte Carlo
- Variational inference: try to approximate a complex, intractable, high-dimensional distribution using a tractable one
 - ▶ Try to minimize the KL divergence
 - ▶ Based on the same math from our EM lecture

CSC412 Teaser: Beyond Clustering

We've seen unsupervised learning algorithms based on two ways of organizing your data

- low-dimensional spaces (dimensionality reduction)
- discrete categories (clustering)

Other ways to organize/model data

- hierarchies
- dynamical systems
- sets of attributes
- topic models (each document is a mixture of topics)

Motifs can be combined in all sorts of different ways

Continuing with machine learning

Courses

- csc413/2516, “Neural Networks and Deep Learning”
- csc412/2506, “Probabilistic Learning and Reasoning”
- csc475, “Reinforcement Learning” (status uncertain)
- Various graduate-level topics courses at UTSG (varies from year to year)

Videos from top ML conferences (NeurIPS, ICML, ICLR, UAI)

- Tutorials and keynote talks are aimed at people with our level of background (know the basics, but not experts in a subfield)
- Try to reproduce results from papers; If authors released code, you can use that as a guide if you get stuck

Lots of excellent free resources available online!